Day 4 — Targeting Latency, Availability, and Rate Limits

> **Activity packet** for your triad. Today's job: set realistic **SLO targets** anchored to user behavior, calculate a **per-hop latency budget**, and design a **rate-limit policy** the team will actually defend.

Where we are in the week

Days 1–3 produced TCD Sections 1–3 (stance, integrations + diagrams, security). Today produces **TCD Section 4 — SLO + rate-limit policy**. The Container diagram from Day 3 is the input: each arrow gets a latency budget; each ingress gets a rate-limit policy; the system as a whole gets an availability target with an error budget.

Inputs

- TCD Sections 1–3 (stance, integration map + diagrams, security/compliance)
- The performance NFRs from PRD Section 7 + their architecture-revised versions
- The Tier Sheet from Week 2 — what user-visible signals will move when we breach an SLO?

SLO vs SLA vs SLI — the vocabulary

A common source of confusion. Get the words right today; everything else follows.

Term	Definition	Audience
SLI (Service Level Indicator)	A measurement: latency, availability, error rate	Operators / engineers
SLO (Service Level Objective)	A target for the SLI (e.g., "p99 < 800ms 99.9% of the time over 30 days")	TPMs, engineering leads
SLA (Service Level Agreement)	A contractual commitment with consequences (often money / credits)	Customer-facing / legal
Error budget	(1 – SLO) over the measurement window — how much "bad" you allow	Engineering leads, TPMs

Today we set **SLOs** (internal targets); SLAs are out of scope unless the customer relationship requires one.

The three SLO categories you must cover

For most features, three SLOs are enough:

1. Latency SLO

- Format: "p ≤ ms % of the time over "
- Example: "p95 ≤ 400ms, 99% of the time over 30 days"
- Anchor to user behavior: how often does the user wait, and at what threshold do they give up?

2. Availability SLO

- Format: "% of requests succeed (non-5xx) over "
- Example: "99.9% over 30 days" (= ~43 minutes downtime/month)
- Anchor: what's the cost of a minute of downtime to the user?

3. Throughput SLO (or rate-limit policy)

- Format: "system handles requests per per "
- Example: "system handles 200 req/sec sustained, 1000 req/sec burst, per tenant"
- Anchor: what abuse / accidental spike are we protecting against?

A TPM doesn't have to *invent* the numbers — they can be derived from existing platform precedent, or from user-behavior data from Week 2's Tier Sheet. But they have to be **defensible**, not aspirational.

The error-budget concept (today's mental model)

If your availability SLO is **99.9%**, your error budget is **0.1%** — about 43 minutes of downtime per 30 days.

The error budget is **load-bearing**:

- If the team is **inside** the budget, they ship features (more risk-tolerant).
- If the team is **near or past** the budget, they prioritize reliability work (more risk-averse).

This converts an abstract SLO into a **decision-making tool**. A team that respects its error budget is doing site reliability engineering, even if they don't call it that.

The latency-budget walk (per-hop math)

If a feature has a top-level SLO of **p95 ≤ 400ms**, that 400ms must be **divided across the hops** in the Container diagram:

```
[Mobile] →(80ms net)→ [API gateway] →(20ms)→ [Reconcile module]
                                                    ↓
                                              (60ms) [Postgres read]
                                                    ↓
                                              (20ms) [Tickets module call]
                                                    ↓
                                               (40ms) [Postgres write]
                                                    ↓
                                               (40ms) [Audit publish]
                                                    ↓
                                            Server-side: 20+60+20+40+40 =
180ms
                                            Network: 80ms each way x2 =
160ms
                                            Total ≈ 340ms  (under 400ms p95)
```

If the math doesn't fit the SLO, **something has to give**:

- Tighter SLO target (less ambitious)
- Reduce hops (cache, async, denormalize)
- Faster individual hops (engineering work)
- Wider SLO percentile (e.g., relax to p90 instead of p95)

The TPM job is to surface this math early — *before* the feature ships and falls short.

Activity 1 — SLO Triage

Format: Triad • **35 min** • Block 1

Purpose

Calibrate on real-world SLO examples — distinguishing realistic targets from cargo-culted ones.

Setup

Each triad receives the **SLO Triage Pack**: 8 SLO statements drawn from real (anonymized) systems. Some are clean. Most have a hidden failure.

The failure modes to hunt for

A well-formed SLO has **three parts**: a percentile, a measurement window, and a defense (rationale). Triage each statement against this checklist:

- **No measurement window** — over what period? always? best case?
- **No percentile (or an average)** — averages hide bimodal distributions.
- **No defense** — a target with no rationale behind the number.
- **Aspirational beyond capability** — a target the team's infra/on-call can't sustain.
- **Mismatched to the user threshold** — a target that's far looser or tighter than the user actually experiences.
- **Confusion with an SLA** — a contractual-sounding commitment where an internal objective belongs.

Triad protocol

1. Triage all 8 (15 min). Identify failures. Mark "clean" if applicable.
2. Rewrite the 5 worst (15 min) to pass the three checks: percentile + window + defense.
3. Identify the one with the most subtle failure (5 min) — discuss in readout.

Readout (60 sec per triad)

> "The most subtle failure was [X] because [why]. Our cleanest rewrite was [example]."

Deliverable

Triaged 8-pack with failure-mode labels and 5 rewrites passing the percentile + window + defense check.

Activity 2 — Set Your Three SLOs

Format: Triad • **40 min** • Block 2

Purpose

Set the three SLOs (latency, availability, throughput / rate limit) for your triad's feature.

Setup

Each triad needs the Week-2 Tier Sheet, the architecture-revised performance NFRs, and any platform precedent the cohort knows. AI optional.

Triad protocol

Step 1 — Latency SLO (15 min)

Use the **NFR template** with these slots:

```
### SLO — Latency
**Measurement:** [what request, what code path]
**Percentile + threshold + window:** p<N> ≤ <X>ms, <PCT>% of the time
over <W> days
**Defense:** [user behavior anchor — Week-2 Tier Sheet quote where
possible]
**Verification:** [what we measure and where]
```

Step 2 — Availability SLO (10 min)

Same template:

```
### SLO — Availability
**Measurement:** [success rate of which requests]
**Target + window:** <N>% over <W> days
**Defense:** [what does a minute of downtime cost the user]
**Verification:** [where the dashboard lives]
**Error budget:** [(1 − target) × window in plain words]
```

Step 3 — Throughput / rate-limit SLO (15 min)

```
### Rate-limit policy
**Per-user limit:** [<X> req/sec, with burst]
**Per-tenant limit:** [<X> req/sec, with burst]
**Global limit:** [<X> req/sec total]
**Defense:** [what abuse / accident this protects against]
**Failure mode:** [what user sees when limited — 429? exponential
backoff?]
**Verification:** [load test or cap monitoring]
```

What "good" looks like

- Each SLO has a **percentile**, a **window**, and a **defense**
- Defense ties to **Week-2 user behavior** where possible
- Targets are **realistic** for current platform capability — not aspirational
- The rate-limit failure mode is **explicit** — most rookie policies skip this

Deliverable

Three SLOs (latency, availability, throughput/rate-limit) using the NFR template, each with percentile/window/defense and an explicit rate-limit failure mode.

Activity 3 — Latency Budget Walk

Format: Triad • **40 min** • Block 3

Purpose

Take the latency SLO and **walk it across the Container diagram** from Day 3 — does the math fit?

Setup

Each triad needs the Container diagram from Day 3, the latency-budget reference card, and the SLO sheet from Activity 2.

Triad protocol

1. **Annotate the Container diagram** (15 min). For each arrow, estimate (or research) the latency contribution. Use:
    - **Network hop**: 20–80ms LAN to internet RTT depending
    - **Database read** (indexed): 5–30ms
    - **Database write** (single-row): 10–50ms
    - **External API call** (synchronous): 50–500ms wildly varies
    - **Async publish** (Kafka/SQS): 1–10ms locally
2. **Sum the path** (10 min). Does the sum fit within the SLO target?
3. **If not, decide what gives** (10 min):
    - Tighter SLO?
    - Reduce hops? (cache, denormalize)
    - Faster hops? (work for engineering)
    - Wider percentile? (p99 → p95 means relaxing the bar)
4. **Flag for engineering conversation** (5 min). What's the question you'd ask the architect tomorrow?

Worked example — FieldPulse reconcile

```
SLO: p95 ≤ 1000ms (modal opens within 1 second)

Path: Mobile → API GW → Reconcile module → Postgres read
                                         → Tickets module
                                         → Postgres write
                                         → Audit publish
                  → response back to Mobile

Estimated:
  – Mobile→API GW round trip:      80ms
  – API GW → Reconcile module:     20ms
  – Postgres read (current state): 30ms
  – Tickets module call (sync):    100ms (their median)
  – Postgres write:                40ms
  – Audit publish (async):         10ms
  – Reconcile → API GW response:   20ms
  – API GW → Mobile response:      80ms
                                   ─────
                                   ~380ms median
                                   p95 likely ~700–900ms
                                   ✓ Fits the 1000ms SLO

Budget remaining for variance: ~100–300ms
Risk: Tickets module call dominates; if their latency degrades,
      we breach the SLO.
```

Deliverable

Annotated Container diagram with per-hop latency estimates, total sum vs the SLO, and a named "what gives" decision if the math doesn't fit.

Activity 4 — Cross-Review + AI Sanity Check

Format: Triad-pair • **45 min** + Wrap • Block 4

Purpose

Cross-review the SLO sheet (same pairing as Day 3 ideal). Use AI to surface what's unrealistic.

Setup

Instructor confirms pairings (re-use Day 3 ideally). Each pair needs both SLO sheets, latency walks, and access to AI for the sanity prompt.

Cross-review protocol (20 min)

Reviewer triad reads the SLO sheet and the latency-budget walk. They ask three questions:

1. **Is the latency target realistic given the platform precedent we know?**
2. **Is the availability target realistic given the team's on-call reality?**
3. **Does the rate-limit policy protect against a real abuse / accident scenario?**

The author triad listens, captures, and decides what to address.

AI sanity-check prompt (15 min)

```
Role: Senior reliability engineer reviewing a feature's SLO sheet.
Context: <paste the SLO sheet + latency-budget walk + rate-limit
policy>
Architecture stance: <paste TCD Section 1>
Task: Identify the top 3 reasons the SLOs as written might not be
sustainable.
Constraints:
  - Use only the information provided; do not invent platform details
  - Be specific about which SLO, which scenario, which condition
  - Suggest a more realistic alternative for each issue
Format: Numbered list, each with: Issue / Scenario / Suggested target.
```

Triad action (10 min)

Decide which AI findings + which peer findings to adopt. Update Section 4. Provenance note.

Deliverable

Updated TCD Section 4 with adopted peer + AI findings, an error-budget consequence statement, and a provenance note for any AI use.

Wrap (last 15 min)

Each triad shares:

- One SLO they would defend hard in front of an engineering lead
- One they're worried is too aspirational (be honest)
- One **error-budget consequence** they would accept (e.g., "if we breach the budget, we freeze new features for a sprint")

The third question is the most important — converting the SLO into a behavior the team will actually adopt.

End-of-day checkpoint

Each triad ends Day 4 with:

- [x] Three **SLOs**: latency, availability, throughput / rate limit
- [x] **Latency-budget walk** annotating the Container diagram
- [x] **Rate-limit policy** with failure-mode behavior named
- [x] **Error-budget consequence** stated — what behavior changes when we breach
- [x] AI provenance note for today
- [x] TCD Section 4 drafted